

EDA and Data Exploration

Group Members: Sai Prabhu (120820618)

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime
import json
from scipy import stats
import seaborn as sns
```

Understanding the Data

Loading and Exploring the Data

```
In [268... ## Loading and Exploring the Data
col_datatypes = {
    "Report Number": str,
    "Local Case Number": str,
    "Agency Name": str,
    "ACRS Report Type": str,
    "Crash Date/Time": str,
    "Route Type": str,
    "Road Name": str,
    "Cross-Street Name": str,
    "Off-Road Description": str,
    "Municipality": str,
    "Related Non-Motorist": str,
    "Collision Type": str,
    "Weather": str,
    "Surface Condition": str,
    "Light": str,
    "Traffic Control": str,
    "Driver Substance Abuse": str,
    "Non-Motorist Substance Abuse": str,
    "Person ID": str,
    "Driver At Fault": str,
    "Injury Severity": str,
    "Circumstance": str,
    "Driver Distracted By": str,
    "Drivers License State": str,
    "Vehicle ID": str,
    "Vehicle Damage Extent": str,
    "Vehicle First Impact Location": str,
    "Vehicle Body Type": str,
    "Vehicle Movement": str,
    "Vehicle Going Dir": str,
    "Speed Limit": int,
    "Driverless Vehicle": str,
    "Parked Vehicle": str,
```

```
"Vehicle Year": int,
"Vehicle Make": str,
"Vehicle Model": str,
"Latitude": float,
"Longitude": float,
"Location": str
}

crash_df = pd.read_csv("Crash_Reporting_-_Drivers_Data.csv", dtype=col_datatype)
print(f"Total Data Entries: {crash_df.shape[0]}")
print(f"Number of Features: {crash_df.shape[1]}")
print(f"Number of Numerical Features: {len(crash_df.select_dtypes(include=['int', 'float']))}")
print(f"Number of Categorical Features: {len(crash_df.select_dtypes(include=['object']))}")
display(crash_df.head())
```

Total Data Entries: 210736
Number of Features: 39
Number of Numerical Features: 4
 -> ['Speed Limit', 'Vehicle Year', 'Latitude', 'Longitude']
Number of Categorical Features: 35
 -> ['Report Number', 'Local Case Number', 'Agency Name', 'ACRS Report Type', 'Crash Date/Time', 'Route Type', 'Road Name', 'Off-Road Description', 'Municipality', 'Related Non-Motorist', 'Collision Type', 'Weather', 'Surface Condition', 'Light', 'Traffic Control', 'Driver Substance Abuse', 'Non-Motorist Substance Abuse', 'Person ID', 'Driver At Fault', 'Injury Severity', 'Circumstance', 'Driver Distracted By', 'Drivers License State', 'Vehicle ID', 'Vehicle Damage Extent', 'Vehicle First Impact Location', 'Vehicle Body Type', 'Vehicle Movement', 'Vehicle Going Dir', 'Driverless Vehicle', 'Parked Vehicle', 'Vehicle Make', 'Vehicle Model', 'Location']

	Report Number	Local Case Number	Agency Name	ACRS Report Type	Crash Date/Time	Route Type	Road Name	
0	MCP3126006X	250037402	MONTGOMERY	Injury Crash	08/21/2025 05:21:00 PM	Maryland (State) Route	NaN	
1	MCP2349001B	250037516	MONTGOMERY	Property Damage Crash	08/22/2025 10:44:00 AM	Interstate (State)	EISENHOWER MEMORIAL HWY	
2	MCP296500BC	250033157	MONTGOMERY	Property Damage Crash	07/25/2025 11:55:00 AM	Bicycle Route	NaN	HAI AN R
3	MCP2159003K	250037509	MONTGOMERY	Property Damage Crash	08/22/2025 10:36:00 AM	Maryland (State) Route	NaN	
4	MCP312900D6	250034573	MONTGOMERY	Property Damage Crash	08/03/2025 02:10:00 PM	NaN	NaN	

5 rows x 39 columns

Parsing and Adjusting the data

Converting Datatypes

In [269...

```
### Lets convert the Crash Date/Time data from string to datetime format

crash_df["Crash Date/Time"] = crash_df["Crash Date/Time"].apply(lambda x: pd.to_datetime(x))

# Generate some new features based on the `Crash Date/Time` Column
crash_df["Season"] = crash_df["Crash Date/Time"].apply(lambda x: "Winter" if x.month in [12, 1, 2] else "Spring" if x.month in [3, 4, 5] else "Summer" if x.month in [6, 7, 8] else "Fall" if x.month in [9, 10, 11] else None)
crash_df["Time of Day"] = crash_df["Crash Date/Time"].apply(lambda x: "Morning" if x.hour in [6, 7, 8] else "Afternoon" if x.hour in [9, 10, 11, 12] else "Evening" if x.hour in [13, 14, 15] else "Night" if x.hour in [16, 17, 18, 19, 20, 21, 22, 23] else None)
display(crash_df.head())
```

	Report Number	Local Case Number	Agency Name	ACRS Report Type	Crash Date/Time	Route Type	Road Name
0	MCP3126006X	250037402	MONTGOMERY	Injury Crash	2025-08-21 17:21:00	Maryland (State) Route	NaN
1	MCP2349001B	250037516	MONTGOMERY	Property Damage Crash	2025-08-22 10:44:00	Interstate (State)	EISENHOWER MEMORIAL HWY
2	MCP296500BC	250033157	MONTGOMERY	Property Damage Crash	2025-07-25 11:55:00	Bicycle Route	NaN
3	MCP2159003K	250037509	MONTGOMERY	Property Damage Crash	2025-08-22 10:36:00	Maryland (State) Route	NaN
4	MCP312900D6	250034573	MONTGOMERY	Property Damage Crash	2025-08-03 14:10:00	NaN	NaN

5 rows x 41 columns

This block allows us to extrapolate further data points, such as season and time of day, for further analysis. In addition, we have changed the data type of `Crash Date/Time` column to be datetime type.

Check how unique values vs total values:

In [270...

```
total_entries = crash_df.shape[0]
unique_df = pd.DataFrame()
unique_df["Column Name"] = crash_df.columns
unique_df["Unique Values"] = unique_df["Column Name"].apply(lambda x: crash_df[x].nunique())
```

```
unique_df["Total Values"] = unique_df["Column Name"].apply(lambda x: total_ent  
unique_df["Is Unique"] = unique_df["Unique Values"] == unique_df["Total Values'  
unique_df
```

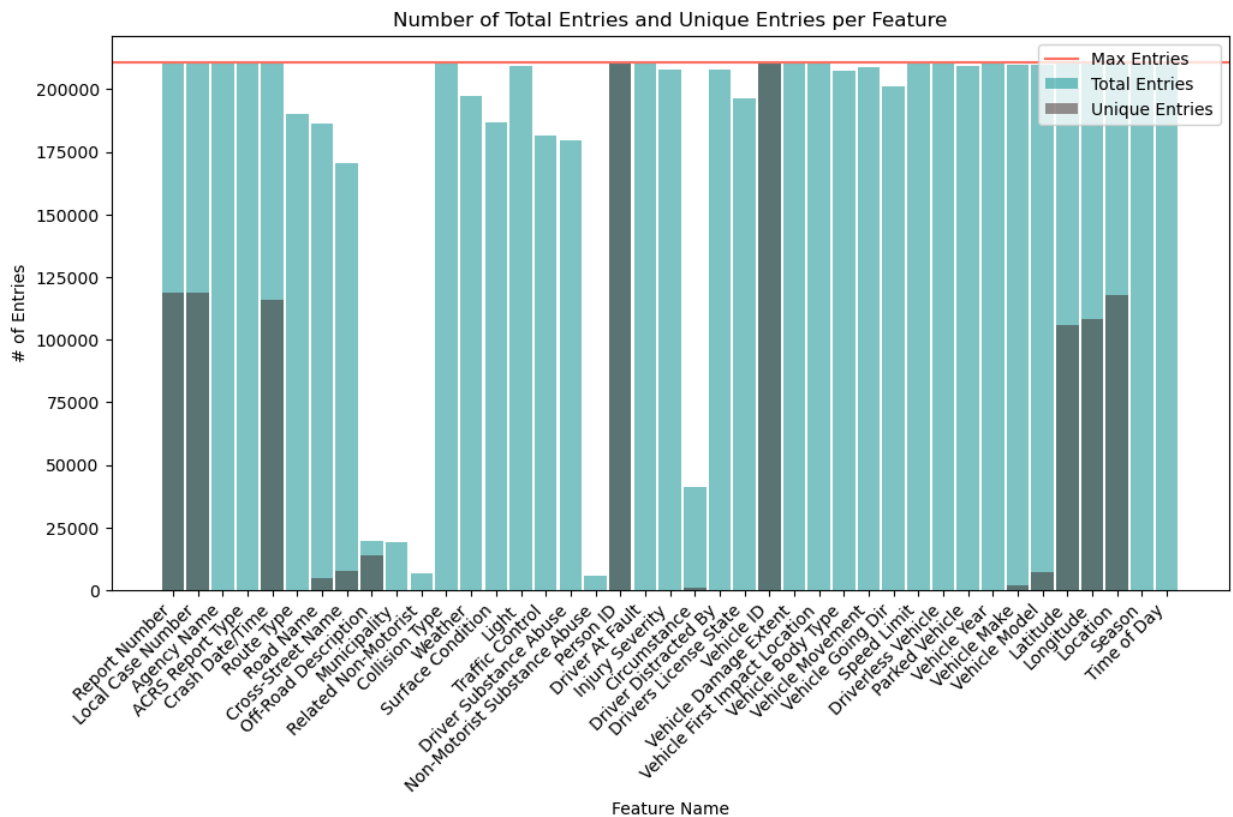
Out [270]:

	Column Name	Unique Values	Total Values	Is Unique
0	Report Number	118843	210736	False
1	Local Case Number	118729	210736	False
2	Agency Name	10	210736	False
3	ACRS Report Type	3	210736	False
4	Crash Date/Time	115943	210736	False
5	Route Type	19	190097	False
6	Road Name	4789	186385	False
7	Cross-Street Name	7536	170288	False
8	Off-Road Description	13978	19597	False
9	Municipality	20	19126	False
10	Related Non-Motorist	28	6879	False
11	Collision Type	28	210151	False
12	Weather	23	197380	False
13	Surface Condition	21	186760	False
14	Light	16	209291	False
15	Traffic Control	33	181648	False
16	Driver Substance Abuse	20	179416	False
17	Non-Motorist Substance Abuse	27	5733	False
18	Person ID	210736	210736	True
19	Driver At Fault	3	210736	False
20	Injury Severity	10	208071	False
21	Circumstance	814	40985	False
22	Driver Distracted By	24	207880	False
23	Drivers License State	81	196386	False
24	Vehicle ID	210736	210736	True
25	Vehicle Damage Extent	12	210420	False
26	Vehicle First Impact Location	33	210580	False
27	Vehicle Body Type	60	207524	False
28	Vehicle Movement	36	209057	False
29	Vehicle Going Dir	10	201211	False
30	Speed Limit	16	210736	False
31	Driverless Vehicle	2	210736	False
32	Parked Vehicle	2	209210	False
33	Vehicle Year	145	210736	False
34	Vehicle Make	1964	209604	False

	Column Name	Unique Values	Total Values	Is Unique
35	Vehicle Model	7183	209562	False
36	Latitude	106074	210736	False
37	Longitude	108265	210736	False
38	Location	117985	210736	False
39	Season	4	210736	False
40	Time of Day	4	210736	False

Let's Graph it!

```
In [323... plt.figure(figsize=(12, 6))
plt.bar(unique_df["Column Name"], unique_df["Total Values"], alpha=0.5, width=0.5)
plt.bar(unique_df["Column Name"], unique_df["Unique Values"], alpha=0.5, width=0.5)
plt.axhline(total_entries, color='#FF6F61', label='Max Entries')
plt.xticks(rotation=45, ha='right')
plt.xlabel("Feature Name")
plt.ylabel("# of Entries")
plt.title("Number of Total Entries and Unique Entries per Feature")
plt.legend(loc="upper right")
plt.show()
```



This information lets us conclude that only the **Vehicle ID** and **Person ID** columns can be used as unique identifiers for our data. Looking at the graph we can also see the density of entries spread across the data. This leads us to think: How many missing

values does each column have and how can we use this information to our advantage?

Understanding the missing values in our data

```
In [272... na_dict = {}  
print("=====[ N/A Statistics ]====")  
print(f"Out of {crash_df.shape[0]} entries, the following number of entries have  
print(f"Total Entries: {crash_df.shape[0]}")  
na_dict = crash_df.isna().sum().astype(int).to_dict()  
sorted_na = dict(sorted(na_dict.items(), key=lambda x: x[1], reverse=True))  
print(json.dumps(sorted_na, indent=2))
```

===== [N/A Statistics] =====

Out of 210736 entries, the following number of entries have no values for the columns]

Total Entries: 210736

```
{
  "Non-Motorist Substance Abuse": 205003,
  "Related Non-Motorist": 203857,
  "Municipality": 191610,
  "Off-Road Description": 191139,
  "Circumstance": 169751,
  "Cross-Street Name": 40448,
  "Driver Substance Abuse": 31320,
  "Traffic Control": 29088,
  "Road Name": 24351,
  "Surface Condition": 23976,
  "Route Type": 20639,
  "Drivers License State": 14350,
  "Weather": 13356,
  "Vehicle Going Dir": 9525,
  "Vehicle Body Type": 3212,
  "Driver Distracted By": 2856,
  "Injury Severity": 2665,
  "Vehicle Movement": 1679,
  "Parked Vehicle": 1526,
  "Light": 1445,
  "Vehicle Model": 1174,
  "Vehicle Make": 1132,
  "Collision Type": 585,
  "Vehicle Damage Extent": 316,
  "Vehicle First Impact Location": 156,
  "Report Number": 0,
  "Local Case Number": 0,
  "Agency Name": 0,
  "ACRS Report Type": 0,
  "Crash Date/Time": 0,
  "Person ID": 0,
  "Driver At Fault": 0,
  "Vehicle ID": 0,
  "Speed Limit": 0,
  "Driverless Vehicle": 0,
  "Vehicle Year": 0,
  "Latitude": 0,
  "Longitude": 0,
  "Location": 0,
  "Season": 0,
  "Time of Day": 0
}
```

From the previous results, we can see that the columns "Non-Motorist Substance Abuse", "Related Non-Motorists", "Municipality", "Off-Road Description", "Circumstance", "Cross-Street Name", "Driver Substance Abuse", and "Traffic Control" have over 25,000 missing values in the dataset. We can conclude that these columns should be reconsidered in our EDA as their omission would allow us to consider more datapoints.

Drawing Conclusions from Our Dataset

Distribution of the frequencies of time of day across crash data

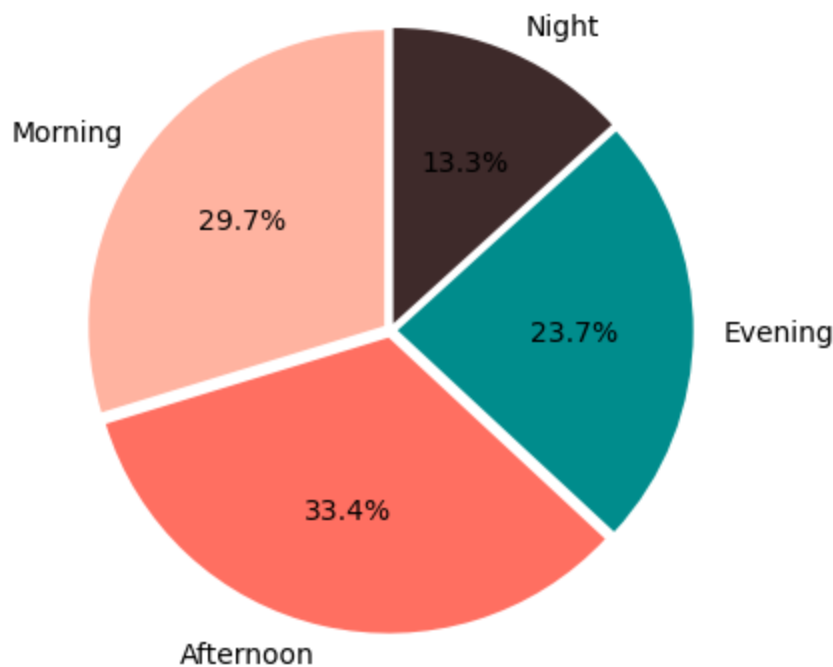
First lets plot the data to see how its distributed

```
In [273... time_freq = pd.DataFrame(columns=["Morning", "Afternoon", "Evening", "Night"])
for time in crash_df["Time of Day"].unique():
    time_freq.loc[0, time] = crash_df[crash_df["Time of Day"] == time].shape[0]

display(time_freq)
plt.pie(time_freq.loc[0,:], explode=[0.025,0.025,0.025,0.025], labels=["Morning", "Afternoon", "Evening", "Night"])
plt.title("The Frequency of Crashes Across Times of the Day")
plt.show()
```

	Morning	Afternoon	Evening	Night
0	62625	70302	49886	27923

The Frequency of Crashes Across Times of the Day



After examining this graph, we have reason to believe there is a connection between crashes and times of the day. To test this hypothesis lets use the chi-squared test:

Using the significance level of $\alpha = 0.05$

- HO: The frequency of crashes is equally distributed across Time of Day
- HA: The frequency of crashes is not equally distributed across Time of Day

```
In [274... chisquare_res = stats.chisquare(time_freq.loc[0,:])

print(f"Chi-square Statistic: {chisquare_res.statistic}")
print(f"p-value: {chisquare_res.pvalue}")
```

Chi-square Statistic: 19553.438045706476
p-value: 0.0

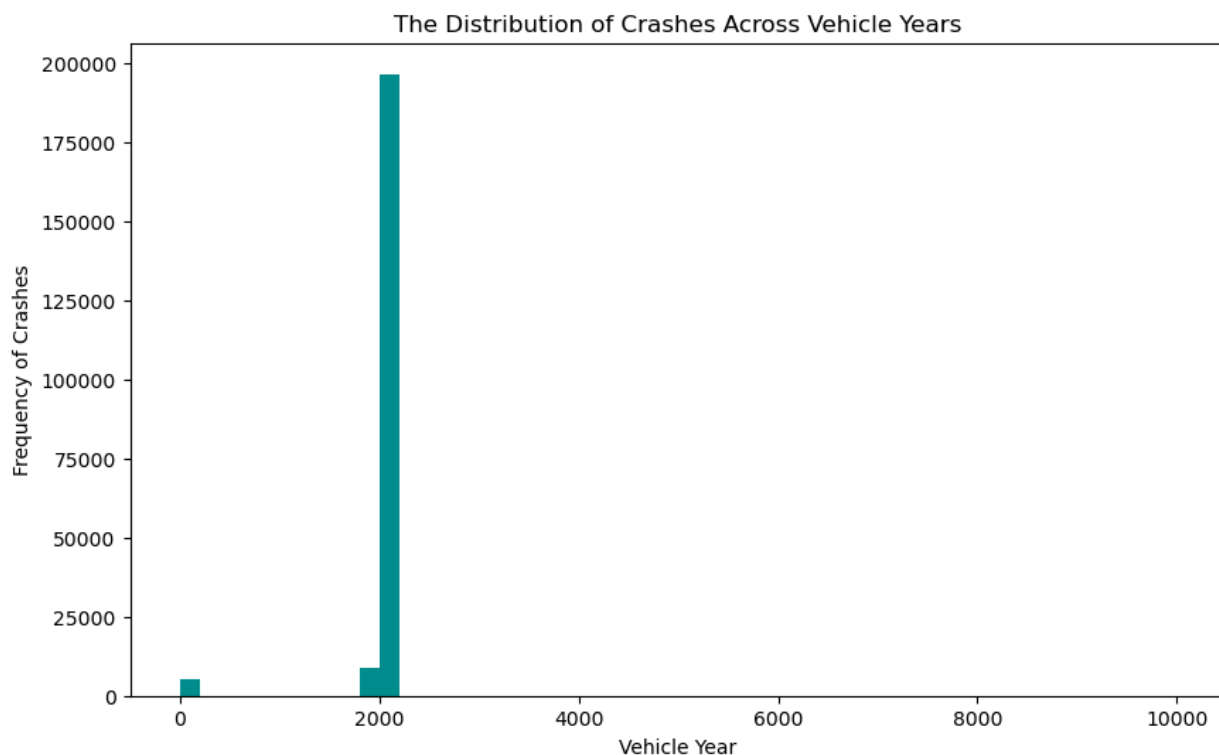
Conclusion

Since this p-value is less than the significance level ($0.0 < 0.05$), we can reject the Null Hypothesis(HO) and conclude that the frequency of crashes cannot be equally distributed across Time of Day

Outlier Testing for Vehicle Years

Let's plot the data for vehicle years to see how its distributed

```
In [275... plt.figure(figsize=(10, 6))
plt.hist(crash_df["Vehicle Year"], bins=50, color="#008C8C")
plt.xlabel("Vehicle Year")
plt.ylabel("Frequency of Crashes")
plt.title("The Distribution of Crashes Across Vehicle Years")
plt.show()
```



Hmm... Something seems to be wrong with the distribution of values for `Vehicle Years`. The first problematic observation that can be made is the hump in the 0 bin. This, for obvious reasons, doesn't make sense for vehicle car years. In addition, we can see that the graph has auto-distributed across a range of $(0, 10000)$. This tells us that there must be some low-frequency outliers towards the 10,000 mark that's spreading the graph thin.

Let's explore this in further detail:

```
In [276... desc_stats = crash_df["Vehicle Year"].describe()
IQR = desc_stats["75%"] - desc_stats["25%"]
lower_bound = int(desc_stats["25%"] - (1.5*IQR))
upper_bound = int(desc_stats["75%"] + (1.5*IQR))
print(f"IQR:{IQR}")
print(f"We declare points outside of the range {lower_bound} <= x <= {upper_bound}")
print("This makes the following unique values outliers:")
vehicle_years = crash_df[(crash_df["Vehicle Year"] > upper_bound) | (crash_df["Vehicle Year"] < lower_bound)]
print(vehicle_years)
```

IQR:10.0

We declare points outside of the range $1991 \leq x \leq 2031$ an outlier

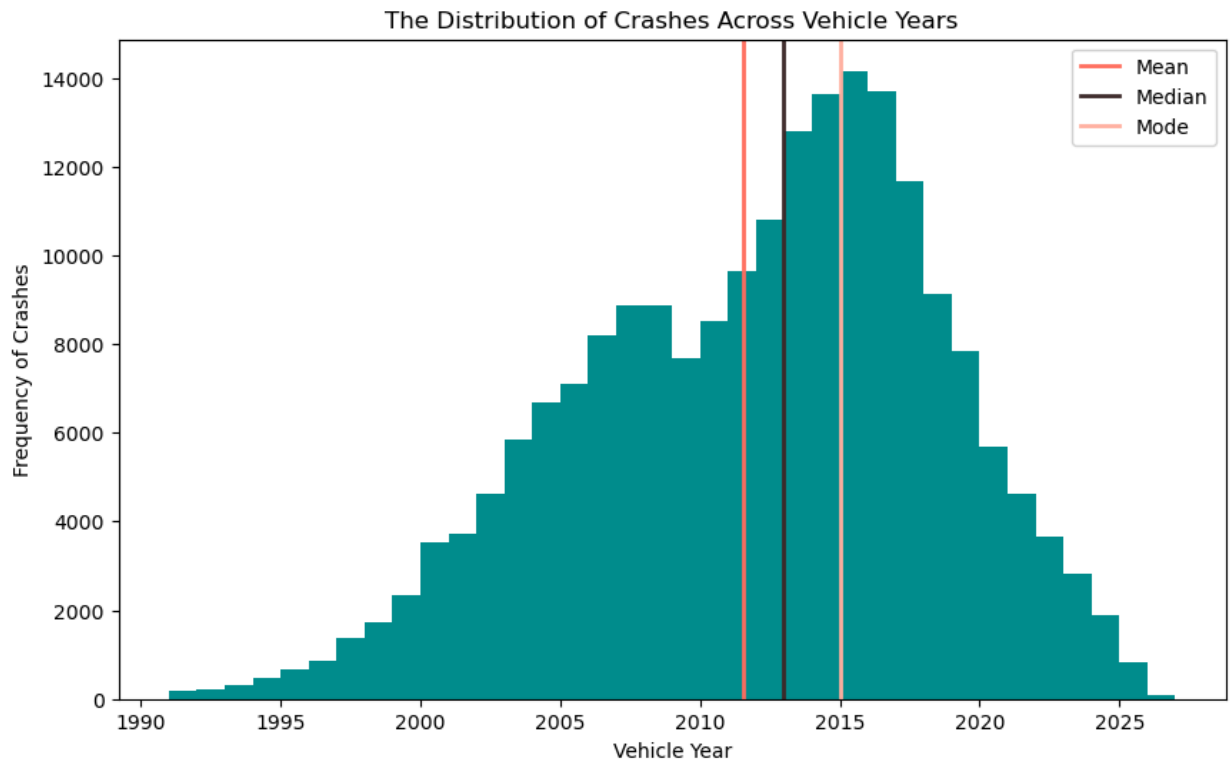
This makes the following unique values outliers:

```
[ 0 1985 1968 1111 1900 1990 1988 1984 1987 1989 1986 1977 9999 99
8888 1901 1981 2915 1983 2101 1966 2033 200 1982 1967 1971 3 1975
1980 1978 1969 201 2918 215 2996 1965 2917 2103 2040 2107 2916 13
1972 1960 1970 15 1911 2105 1 1938 1979 2991 1949 2997 202 1976
97 3013 2914 5005 2911 2201 1140 1961 1005 1947 2099 1014 2100 1955
2208 1974 2041 2200 7817 1930 8 1963 2048 2104 1015 198 2919 2102
2055 3003 8008 2204 3863 2912 2 999 1008 1946 14 1959 1025 2998
1234 1012 4 2205 1000 9000 3012 2109 2222 1957]
```

Looking at the data, we can compute an IQR of 10 (2016 - 2006). This gives us a lower_bound and upper_bound of 1991 and 2031 respectively. When we calculate the possible outlier points, however, we allow implausible values to persist in our data. To elaborate, an upper_bound of 2031 doesn't make sense to describe this feature, since the maximum possible value for Vehicle Year is 2027. For these reasons we might want to implement a manual cap for the upper bounds and keep the lower bounds the same.

How about we implement a hardcap for the upperbounds?

```
In [264... filtered_values = crash_df[(crash_df["Vehicle Year"]<=2027) & (crash_df["Vehic
#print(sorted(filtered_values.unique()))
plt.figure(figsize=(10, 6))
plt.hist(filtered_values, bins=36, range=(1991,2027), color="#008C8C")
plt.axvline(filtered_values.mean(), label="Mean", color="#FF6F61", lw=2.0)
plt.axvline(filtered_values.median(), label="Median", color="#3E2A2A", lw=2.0)
plt.axvline(filtered_values.mode()[0], label="Mode", color="#FFB3A0", lw=2.0)
plt.xlabel("Vehicle Year")
plt.ylabel("Frequency of Crashes")
plt.title("The Distribution of Crashes Across Vehicle Years")
plt.legend()
plt.show()
```



With this showing promising results, let's drop any rows that fall outside of the lower, upper range

```
In [265... out_of_bounds_idx = crash_df[(crash_df["Vehicle Year"]>2027) | (crash_df["Vehicle Year"]<2008)]
crash_df.drop(index=out_of_bounds_idx,axis=0,inplace=True)
print(f"Entries Left: {crash_df.shape[0]}, Entries Deleted: {total_entries-crash_df.shape[0]}")
```

Entries Left: 204692, Entries Deleted: 6044

Conclusion

We can effectively select our lower and upper bounds for outliers by using a modified version of IQR outlier selection. We choose to use the lower bound from IQR because it provides an accurate representation of the data. For the upper bounds, however, we choose to select the maximum possible valid year for Vehicle Years. This allows us to pick the best possible range for outliers.

In addition, we can conclude that the distribution of Vehicle Years is left-skewed since the **Mode > Median > Mean**. Furthermore, we can see that the data is bimodal with peaks at 2008 and 2016

Correlation Testing Between Categorical Data

In order to explore the relationships between our categorical features, we might opt to use a correlation heatmap. This type of graph can display the effective relationships between variables, which variables might be redundant, and allows us to pick the most useful features for future exploration.

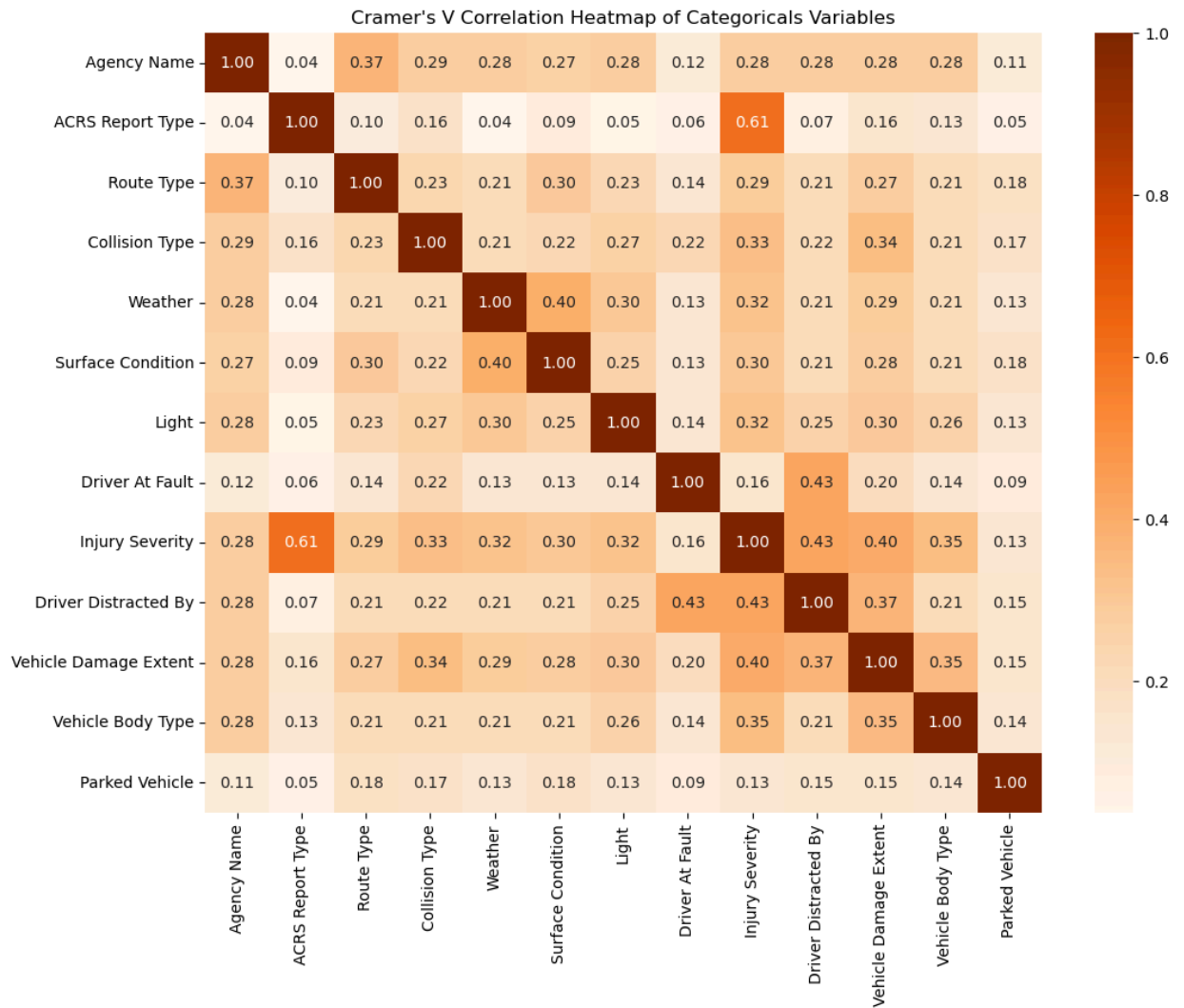
Lets start with creating our heatmap!

```
In [304... # ['Report Number', 'Local Case Number', 'Agency Name', 'ACRS Report Type', 'C
# `Non-Motorist Substance Abuse`, `Related Non-Motorists`, `Municipality`
cat_cols = ["Agency Name", "ACRS Report Type", "Route Type", "Collision Type",
cat_df = crash_df[cat_cols]
heat_map = pd.DataFrame(index=cat_cols, columns=cat_cols)

for x in cat_cols:
    for y in cat_cols:
        contingency_table = stats.contingency.crosstab(cat_df[x].astype(str),
        #display(contingency_table.count)
        heat_map.loc[x,y] = stats.contingency.association(contingency_table.co
#display(heat_map)
```

Let's plot it!

```
In [320... plt.figure(figsize=(12, 9))
sns.heatmap(heat_map.astype(float), annot=True, cmap='Oranges', fmt='.2f')
plt.title("Cramer's V Correlation Heatmap of Categoricals Variables")
plt
plt.show()
```



```
In [318... rankings_dict = {}
for row_idx in range(len(heat_map.columns)):
    for col_idx in range(len(heat_map.columns)):
        if col_idx >= row_idx:
            continue
        rankings_dict[(heat_map.columns[row_idx], heat_map.columns[col_idx])] = heat_map.values[row_idx, col_idx]
sorted_dict = dict(sorted(rankings_dict.items(), key=lambda x: x[1], reverse=True))

for key in sorted_dict:
    print(f" Columns {key[0]} and {key[1]} have a Cramer's coefficient of {sorted_dict[key]}")
```

Columns Injury Severity and ACRS Report Type have a Cramer's coefficient of 0.6073613282429203

Columns Driver Distracted By and Injury Severity have a Cramer's coefficient of 0.4260253437498754

Columns Driver Distracted By and Driver At Fault have a Cramer's coefficient of 0.4250439454866494

Columns Vehicle Damage Extent and Injury Severity have a Cramer's coefficient of 0.40322529203386015

Columns Surface Condition and Weather have a Cramer's coefficient of 0.39742403334923165

Columns Route Type and Agency Name have a Cramer's coefficient of 0.3717892691611726

Columns Vehicle Damage Extent and Driver Distracted By have a Cramer's coefficient of 0.36667599489768077

Columns Vehicle Body Type and Vehicle Damage Extent have a Cramer's coefficient of 0.3471805246865967

Columns Vehicle Body Type and Injury Severity have a Cramer's coefficient of 0.3460699835020803

Columns Vehicle Damage Extent and Collision Type have a Cramer's coefficient of 0.341614835791802

Columns Injury Severity and Collision Type have a Cramer's coefficient of 0.3063880123633066

Columns Injury Severity and Light have a Cramer's coefficient of 0.31910589273140355

Columns Injury Severity and Weather have a Cramer's coefficient of 0.31851509910565506

Columns Injury Severity and Surface Condition have a Cramer's coefficient of 0.30275377747265625

Columns Light and Weather have a Cramer's coefficient of 0.2992116123950457

Columns Surface Condition and Route Type have a Cramer's coefficient of 0.29730165808160924

Columns Vehicle Damage Extent and Light have a Cramer's coefficient of 0.29663796236591616

Columns Vehicle Damage Extent and Weather have a Cramer's coefficient of 0.2933726549656194

Columns Injury Severity and Route Type have a Cramer's coefficient of 0.2917369320247345

Columns Collision Type and Agency Name have a Cramer's coefficient of 0.28600894327846754

Columns Vehicle Damage Extent and Surface Condition have a Cramer's coefficient of 0.2843526838190477

Columns Vehicle Damage Extent and Agency Name have a Cramer's coefficient of 0.2841310361957817

Columns Vehicle Body Type and Agency Name have a Cramer's coefficient of 0.283553355988402

Columns Light and Agency Name have a Cramer's coefficient of 0.2833799968907509

Columns Driver Distracted By and Agency Name have a Cramer's coefficient of 0.2831731884755379

Columns Weather and Agency Name have a Cramer's coefficient of 0.28227159610381874

Columns Injury Severity and Agency Name have a Cramer's coefficient of 0.2822712833417366

Columns Vehicle Damage Extent and Route Type have a Cramer's coefficient of 0.2746722152902895

Columns Light and Collision Type have a Cramer's coefficient of 0.26960928608123863

Columns Surface Condition and Agency Name have a Cramer's coefficient of 0.2681902392746294

Columns Vehicle Body Type and Light have a Cramer's coefficient of 0.25789151

439962416

Columns Driver Distracted By and Light have a Cramer's coefficient of 0.25459

86222305506

Columns Light and Surface Condition have a Cramer's coefficient of 0.25095849

93267835

Columns Collision Type and Route Type have a Cramer's coefficient of 0.234709

0573623787

Columns Light and Route Type have a Cramer's coefficient of 0.230856754433159

73

Columns Surface Condition and Collision Type have a Cramer's coefficient of

0.2229487191170923

Columns Driver At Fault and Collision Type have a Cramer's coefficient of 0.2

1977474281728115

Columns Driver Distracted By and Collision Type have a Cramer's coefficient o

f 0.2151216029752219

Columns Vehicle Body Type and Driver Distracted By have a Cramer's coefficient

t of 0.2148589177690421

Columns Weather and Collision Type have a Cramer's coefficient of 0.214798935

31005165

Columns Vehicle Body Type and Route Type have a Cramer's coefficient of 0.213

5536005548579

Columns Driver Distracted By and Route Type have a Cramer's coefficient of 0.

21248617208132925

Columns Weather and Route Type have a Cramer's coefficient of 0.2114198416725

855

Columns Driver Distracted By and Weather have a Cramer's coefficient of 0.211

33171508346665

Columns Vehicle Body Type and Weather have a Cramer's coefficient of 0.210941

8126525307

Columns Driver Distracted By and Surface Condition have a Cramer's coefficient

t of 0.20982842034000648

Columns Vehicle Body Type and Surface Condition have a Cramer's coefficient o

f 0.2097990033902868

Columns Vehicle Body Type and Collision Type have a Cramer's coefficient of

0.2064619824365677

Columns Vehicle Damage Extent and Driver At Fault have a Cramer's coefficient

of 0.1991142740787882

Columns Parked Vehicle and Route Type have a Cramer's coefficient of 0.180554

8426806838

Columns Parked Vehicle and Surface Condition have a Cramer's coefficient of

0.17704740438289215

Columns Parked Vehicle and Collision Type have a Cramer's coefficient of 0.16

633228795815408

Columns Collision Type and ACRS Report Type have a Cramer's coefficient of 0.

1629722953095272

Columns Vehicle Damage Extent and ACRS Report Type have a Cramer's coefficient

t of 0.16147182250830472

Columns Injury Severity and Driver At Fault have a Cramer's coefficient of 0.

15869065130787552

Columns Parked Vehicle and Vehicle Damage Extent have a Cramer's coefficient

of 0.1502373580743102

Columns Parked Vehicle and Driver Distracted By have a Cramer's coefficient o

f 0.14940264531840508

Columns Vehicle Body Type and Driver At Fault have a Cramer's coefficient of

0.14432084188402064

Columns Driver At Fault and Route Type have a Cramer's coefficient of 0.14029

33972994384

Columns Parked Vehicle and Vehicle Body Type have a Cramer's coefficient of

0.13739136099204322

Columns Driver At Fault and Light have a Cramer's coefficient of 0.1370321056

6037421

Columns Driver At Fault and Weather have a Cramer's coefficient of 0.13391069249164944

Columns Driver At Fault and Surface Condition have a Cramer's coefficient of 0.13356061430448832

Columns Parked Vehicle and Weather have a Cramer's coefficient of 0.13306842927256046

Columns Parked Vehicle and Injury Severity have a Cramer's coefficient of 0.13169595548250015

Columns Vehicle Body Type and ACRS Report Type have a Cramer's coefficient of 0.13075202602671712

Columns Parked Vehicle and Light have a Cramer's coefficient of 0.1304496199218196

Columns Driver At Fault and Agency Name have a Cramer's coefficient of 0.12068442664487372

Columns Parked Vehicle and Agency Name have a Cramer's coefficient of 0.11401930307881325

Columns Route Type and ACRS Report Type have a Cramer's coefficient of 0.10307957745502686

Columns Surface Condition and ACRS Report Type have a Cramer's coefficient of 0.09309494404199042

Columns Parked Vehicle and Driver At Fault have a Cramer's coefficient of 0.08808281817217256

Columns Driver Distracted By and ACRS Report Type have a Cramer's coefficient of 0.06958208301772502

Columns Driver At Fault and ACRS Report Type have a Cramer's coefficient of 0.05916592353108591

Columns Parked Vehicle and ACRS Report Type have a Cramer's coefficient of 0.04585373763914654

Columns Light and ACRS Report Type have a Cramer's coefficient of 0.0453495564035588

Columns ACRS Report Type and Agency Name have a Cramer's coefficient of 0.03849410578016047

Columns Weather and ACRS Report Type have a Cramer's coefficient of 0.03806903368852825

Conclusion

From this heat map and ranking of information we can conclude which categorical features would be correlated. From our findings, we can conclude from the (Injury Severity, ACRS Report Type), (Driver Distracted, Injury Severity), (Driver Distracted, Driver At Fault), (Damage Extent, Injury Severity), (Surface Condition, Weather), (Route Type, Agency Name), (Vehicle Damage Extent, Driver Distracted) are strongly linked as they have a Cramer's $V > 0.35$.

It's important to note that we cannot conclude that these variables are positively or negatively correlated, only that knowing one can be a meaningful way of predicting the other.

